

Universidad Politécnica de Chiapas



Nombres del alumno:

Escobar Nuricumbo Heber Alexander

Matrícula:

243691

Proyecto:

Sistema de Gestión de Clínica Veterinaria

Materia:

Base de Datos Avanzadas

19 de Abril del 2026.

Suchiapa, Chiapas.

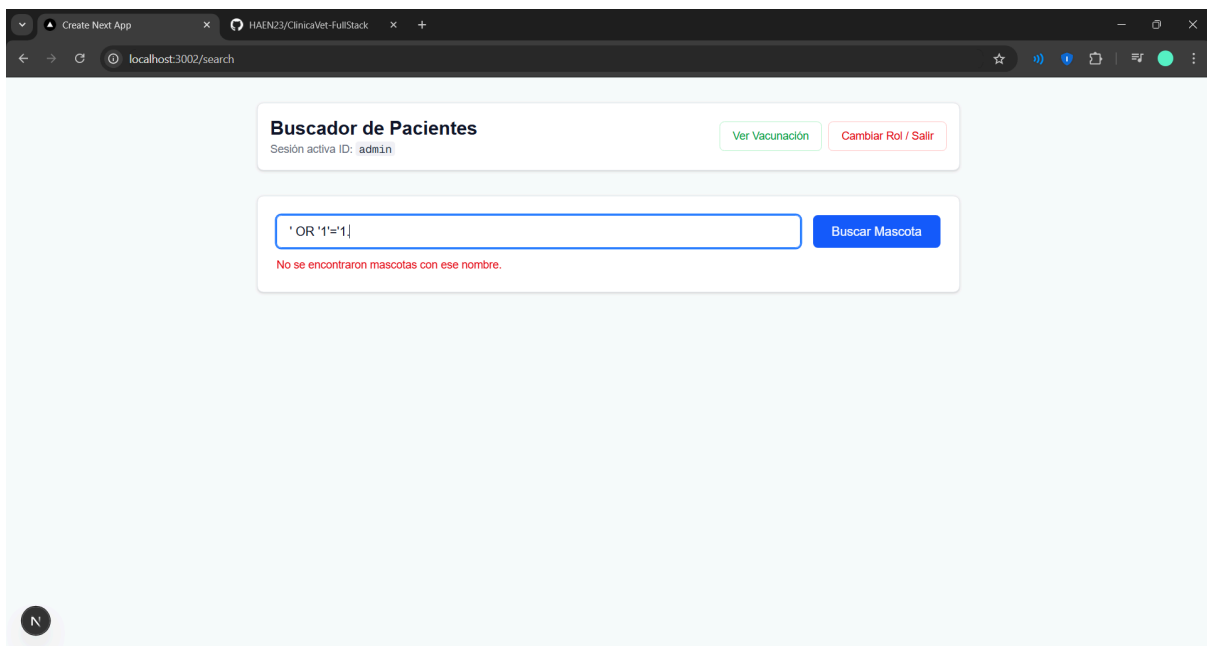
Cuaderno de Ataques y Fortalecimiento

1. Ataque de Inyección SQL

Vulnerabilidad intentada: Inyección basada en errores lógicos (' OR '1'='1')

Punto de entrada: Buscador de mascotas.

Descripción del intento: Aquí el atacante rompe la condición 'WHERE'. En vez de buscar un nombre en específico, la base de datos evalúa la condición completa. Además (1=1) siempre es verdadera, la base de datos devuelve todos los registros de la tabla.

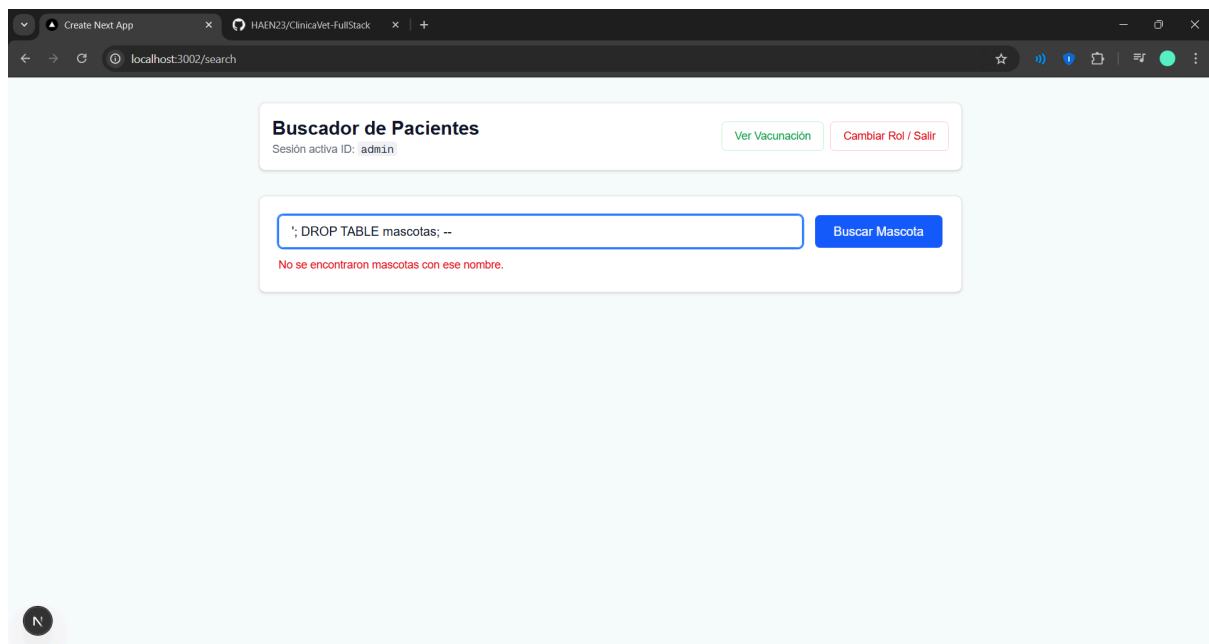


Mecanismo de Defensa: Aquí no pasa esto ya que Prisma es un intermediario, toma lo que escribe el usuario y lo toma como un dato (un nombre) en vez de una instrucción, en pocas palabras solo busca un perro cuyo nombre literal es ' OR '1'='1'. Como no existe, no devuelve nada.

Vulnerabilidad intentada: Inyección basada en errores lógicos (; DROP TABLE mascotas; --)

Punto de entrada: Buscador de mascotas

Descripción del intento: Aquí el atacante utiliza el punto y como para dar a entender a la base de datos que una instrucción ha terminado y puede empezar otra, aquí el atacante cierra la consulta real del código y anexa una orden destructiva la cual es 'DROP TABLE' (en este caso) que causaría borrar todo y los dos guiones al final dice a la base de datos que convierta en un "comentario" inofensivo cualquier código legítimo que el backend haya puesto después, para que no cause errores de sintaxis.

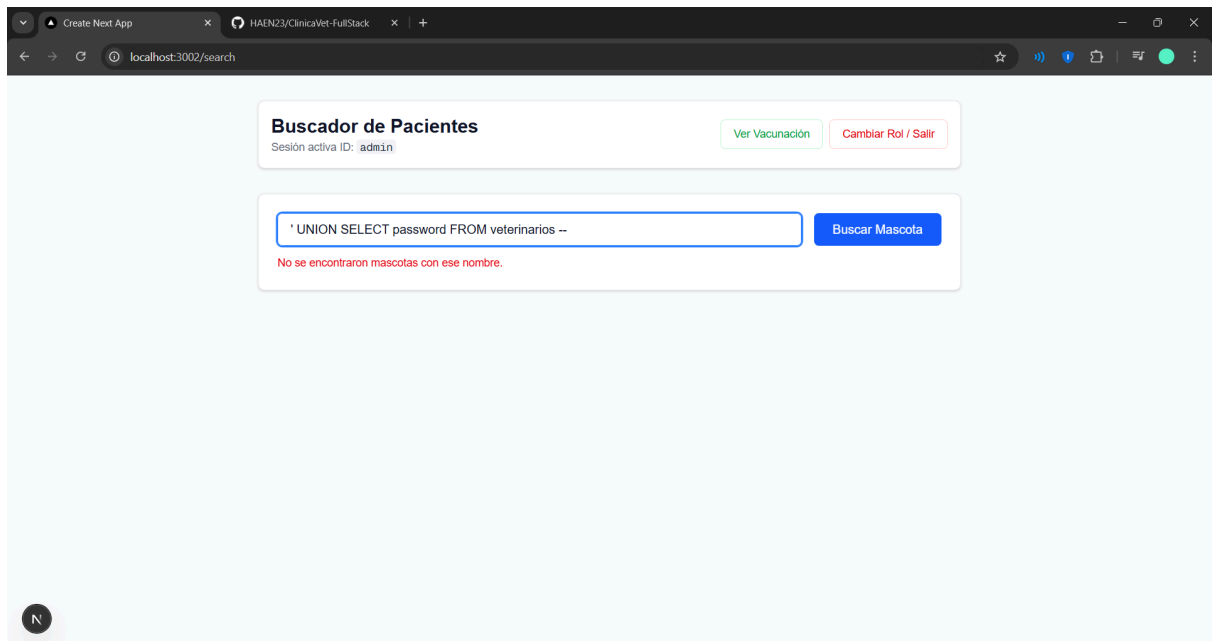


Mecanismo de Defensa: Igual que el otro caso Prisma interviene aquí porque solo busca un perro con ese nombre, en pocas palabras ignora el ataque, gracias al hardening o sea el proceso que protege el sistema gestor de la base de datos contra accesos no autorizados, corrupción de datos y ataques.

Vulnerabilidad intentada: Inyección basada en errores lógicos (' UNION SELECT password FROM veterinarios --')

Punto de entrada: Buscador de mascotas.

Descripción del intento: Aquí el atacante usa la palabra 'UNION' en SQL sirve para fusionar los resultados de dos consultas distintas para “confundir” al sistema para que diera las contraseñas de los veterinarios y lo muestra en la pantalla del frontend como si fueran resultados normales de las búsqueda.



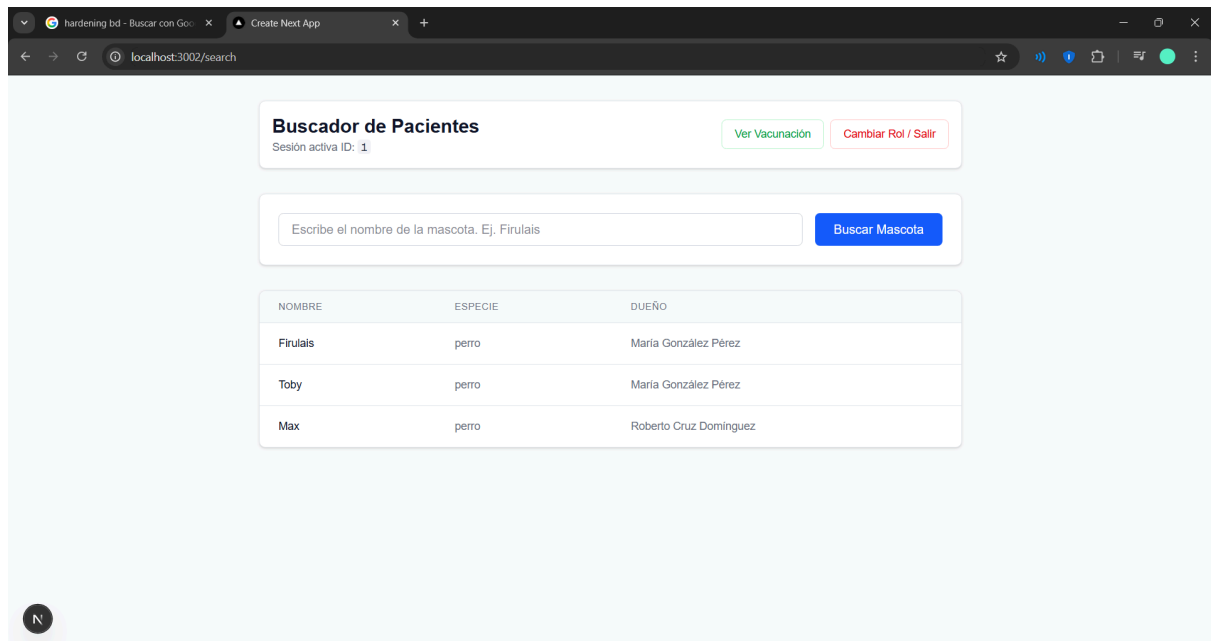
Mecanismo de Defensa: Aquí el sistema agarra la palabra “UNION” como si fuera un apellido del usuario que se intenta buscar, como los dos casos anteriores esto ya que se usa el hardening.

2.- Demostración de RLS en acción

Objetivo: Demostrar que dos usuarios con el mismo nivel de privilegios (Rol Veterinario) obtienen conjuntos de datos distintos al hacer la misma consulta, garantizando el aislamiento de los pacientes.

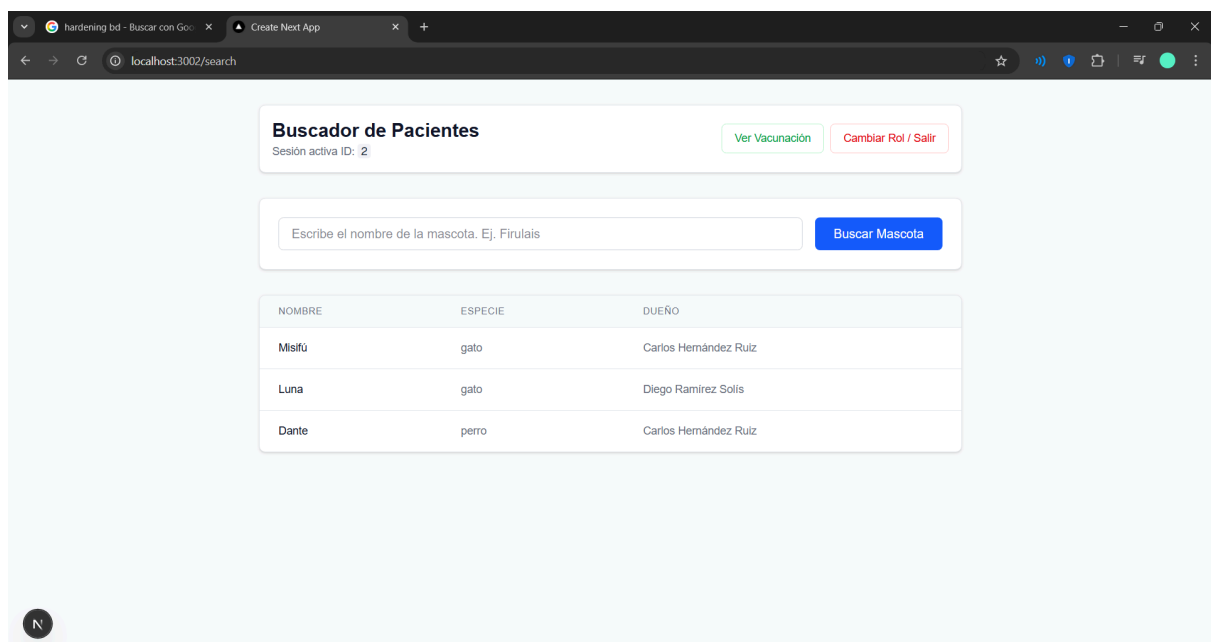
Prueba con Veterinario 1

Al iniciar sesión con el ID de un veterinario y buscar en el catálogo general, el sistema sólo devuelve las mascotas que le corresponden.



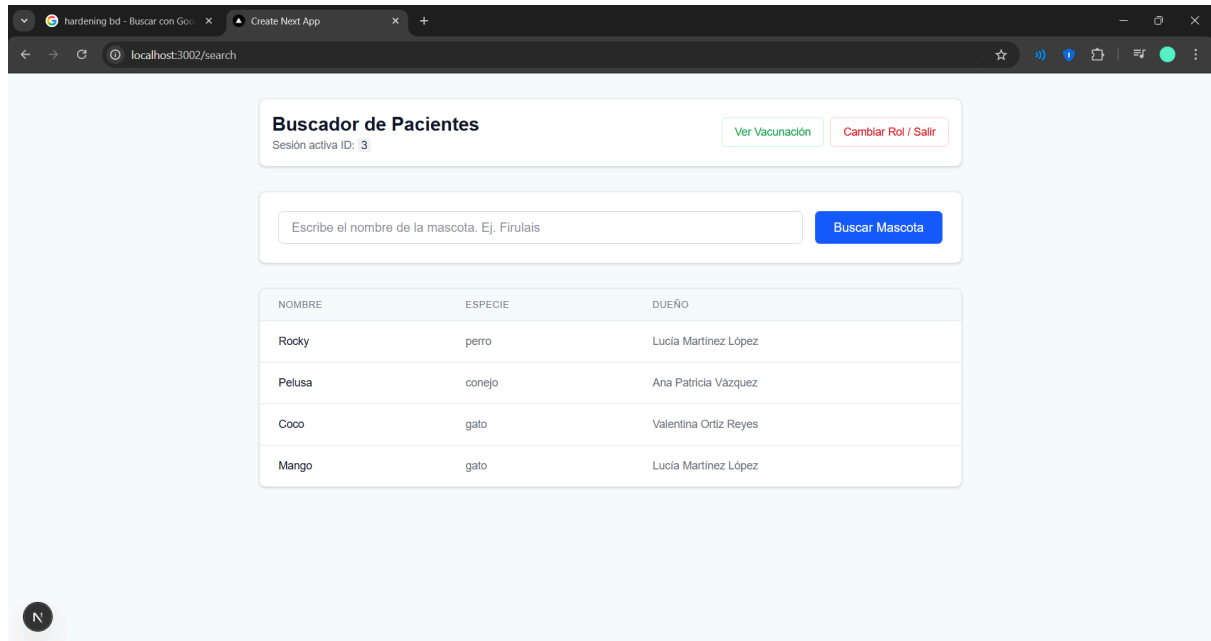
Prueba con Veterinario 2

Al iniciar sesión con un ID distinto y realizar la misma búsqueda, los resultados cambian completamente.



Prueba con Veterinario 3

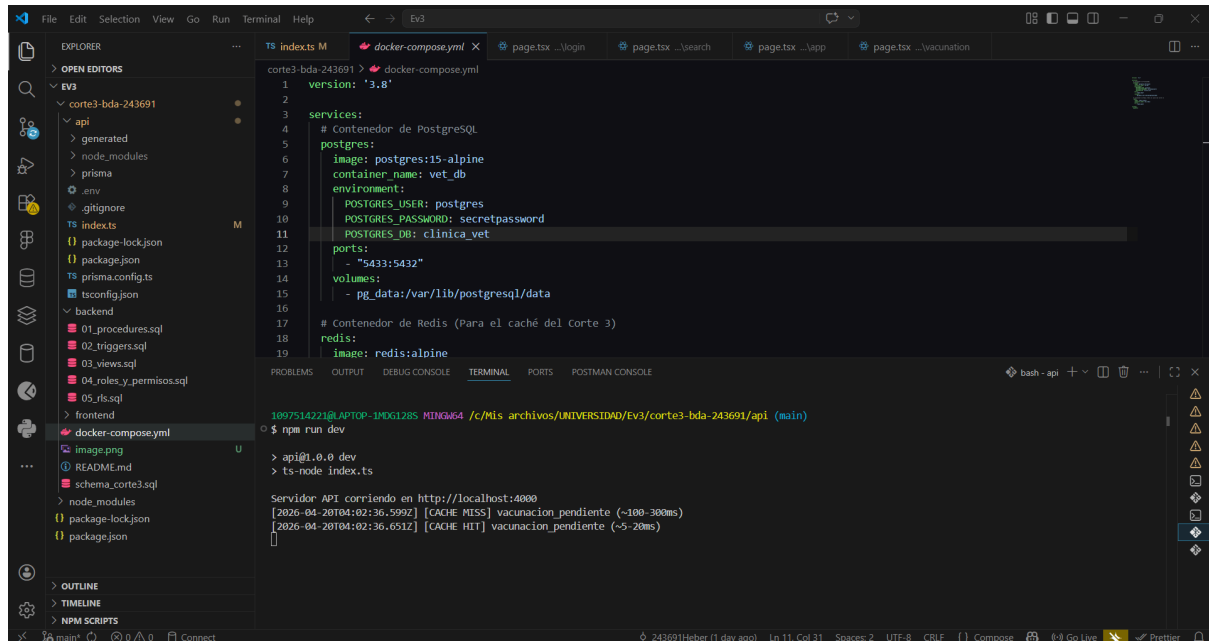
Igual que los otros dos veterinarios sus pacientes son totalmente diferentes



Explicación:

Aquí se ve en acción el RLS (Row-Level Security) trabajando con la tabla de `vet_atienede_mascota`, lo que hace es dependiendo el id del veterinario les muestra sus pacientes correspondientes porque la base de datos reconoce 'quien' está preguntando, y filtra la información desde la raíz usando la tabla relacionada, haciendo que no aparezca pacientes que no sean los correspondidos al veterinario.

Demostración de caché Redis funcionando



The screenshot shows the VS Code interface with the Explorer on the left, the Docker Compose file open in the center, and the terminal at the bottom. The Docker Compose file defines two services: 'postgres' and 'redis'. The 'postgres' service uses the 'postgres:15-alpine' image and is configured with environment variables for user, password, and database name. The 'redis' service uses the 'redis:alpine' image. The terminal shows the command 'npm run dev' being executed, and the output indicates that the API is running on http://localhost:4000. The first log entry shows a 'CACHE MISS' for the 'vacunacion_pendiente' query, with a latency of approximately 100-300ms.

```
version: '3.8'

services:
  # Contenedor de PostgreSQL
  postgres:
    image: postgres:15-alpine
    container_name: vet_db
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: secretpassword
      POSTGRES_DB: clinica_vet
    ports:
      - "5433:5432"
    volumes:
      - pg_data:/var/lib/postgresql/data

  # Contenedor de Redis (Para el caché del Corte 3)
  redis:
    image: redis:alpine
```

```
1097514221@APTOP-JHGX1285 MINGW64 /c/Mis archivos/UNIVERSIDAD/Ev3/corte3-bda-243691/api (main)
$ npm run dev

> api@1.0.0 dev
> ts-node index.ts

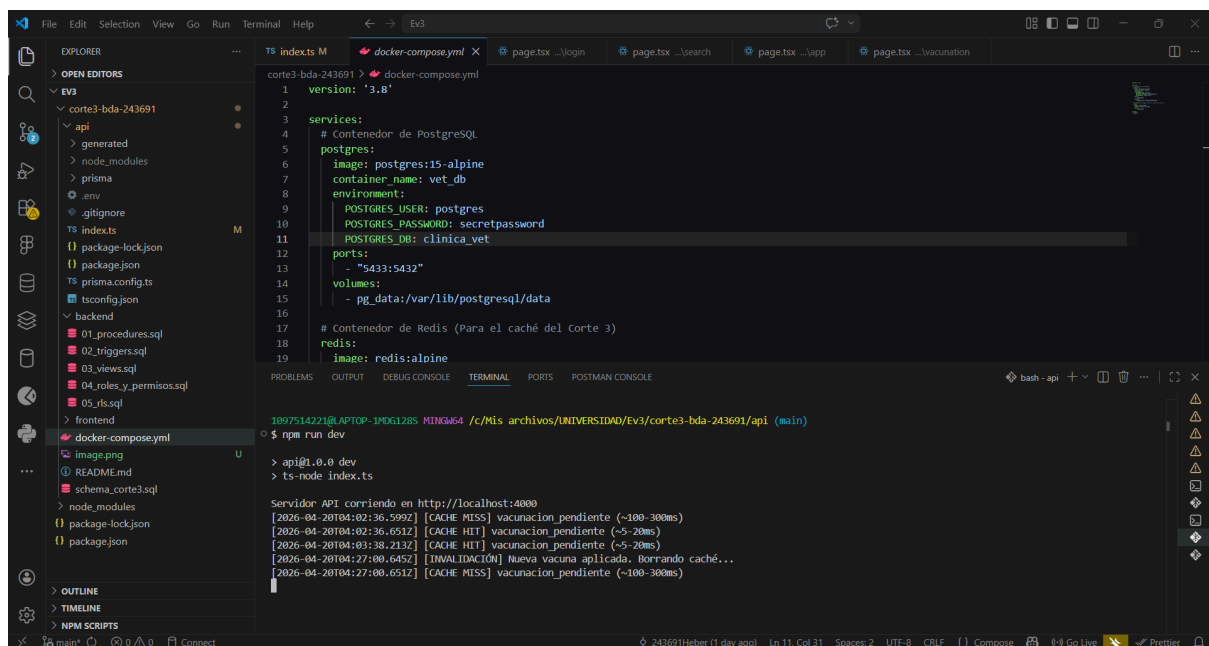
Servidor API corriendo en http://localhost:4000
[2026-04-20T04:02:36.599Z] [CACHE MISS] vacunacion_pendiente (~100-300ms)
[2026-04-20T04:02:36.651Z] [CACHE HIT] vacunacion_pendiente (~5-20ms)
```

1. Primera Consulta (Cache MISS)

Al entrar por primera vez a la vista de vacunación pendiente, Redis no tiene los datos en memoria. La petición viaja hasta PostgreSQL, tardando la latencia típica de una base de datos.

2. Segunda Consulta Inmediata (Cache HIT)

Al recargar la página, la petición ya no llega a PostgreSQL. Redis intercepta la consulta y la devuelve desde la memoria RAM, bajando drásticamente el tiempo de respuesta.



This screenshot is similar to the first one, but the terminal output shows the second log entry. The second log entry shows a 'CACHE HIT' for the 'vacunacion_pendiente' query, with a latency of approximately 5-20ms, indicating that the data was retrieved from the Redis cache.

```
1097514221@APTOP-JHGX1285 MINGW64 /c/Mis archivos/UNIVERSIDAD/Ev3/corte3-bda-243691/api (main)
$ npm run dev

> api@1.0.0 dev
> ts-node index.ts

Servidor API corriendo en http://localhost:4000
[2026-04-20T04:02:36.599Z] [CACHE MISS] vacunacion_pendiente (~100-300ms)
[2026-04-20T04:02:36.651Z] [CACHE HIT] vacunacion_pendiente (~5-20ms)
[2026-04-20T04:03:38.213Z] [CACHE HIT] vacunacion_pendiente (~5-20ms)
[2026-04-20T04:27:00.645Z] [INVALIDACIÓN] Nueva vacuna aplicada. Borrando caché...
[2026-04-20T04:27:00.651Z] [CACHE MISS] vacunacion_pendiente (~100-300ms)
```

3. Invalidación de Caché

Al registrar una nueva vacuna en el sistema, el backend borra el registro de Redis para evitar mostrar datos desactualizados (Stale Data).